Algoritmi e Strutture Dati – Cap. 3.3: Alberi

Supporto teorico per le implementazioni in C

Riferimento: Demetrescu, Finocchi, Italiano *Algoritmi e Strutture Dati*

1 Introduzione e Terminologia

In informatica, le strutture ad albero permettono di modellare collezioni di dati caratterizzate da relazioni gerarchiche (es. file system, alberi genealogici, gerarchie di espressioni).

Un **albero radicato** è formalmente definito come una coppia $T = (N, A)$ costituita da un insieme N di nodi e da un insieme $A \subseteq N \times N$ di coppie di nodi, dette **archi**. In un albero:

- Esiste un unico nodo, detto **radice**, privo di padre.
- Ogni altro nodo v ha un solo genitore (o padre) u tale che $(u, v) \in A$.
- Ogni nodo è raggiungibile dalla radice seguendo un unico cammino orientato.
- Un nodo u può avere zero o più **figli** v tali che $(u, v) \in A$. Il numero di figli di un nodo è detto **grado** del nodo ($\deg(u)$).
- Un nodo senza figli è detto **foglia**, mentre i nodi intermedi sono detti **nodi interni**.
- Nodi con lo stesso genitore si dicono **fratelli**.

Nel contesto dei nostri algoritmi in C, considereremo principalmente alberi **binari ordinati**, in cui ogni nodo possiede al più due figli, distinti in **figlio sinistro** (`left`) e **figlio destro** (`right`).

1.1 Profondità e Altezza

- **Profondità (o livello)** di un nodo: il numero di archi da attraversare a partire dalla radice (la radice ha profondità 0).
- **Altezza** di un albero: la massima profondità a cui si trova una foglia.

2 Struttura Dati e Rappresentazione Collegata in C

A differenza delle liste collegate lineari, gli alberi richiedono una rappresentazione non lineare (gerarchica), basata su record e puntatori.

Per garantire efficienza nelle operazioni di navigazione, ogni nodo memorizza:

1. Il campo dati (`data` o `info`).
2. Il puntatore al figlio sinistro (`left`).
3. Il puntatore al figlio destro (`right`).
4. Il puntatore al genitore (`parent`), che consente di risalire verso la radice in tempo costante $\mathbf{O(1)}$ anziché dover percorrere l'albero dall'alto.

2.1 Operazioni Fondamentali sul Nodo

- **Creazione (`create_node`):** Alloca dinamicamente un nuovo nodo tramite `malloc`, inizializzando i puntatori a `NULL`. Complessità: $O(1)$.
- **Cancellazione (`free_tree`):** Elimina ricorsivamente ogni nodo dell'albero (in post-order). Complessità: $O(n)$.
- **Misurazione della dimensione (`num_nodi`):** Conta ricorsivamente i nodi dell'albero ($1 +$ sinistra $+$ destra). Complessità: $O(n)$, poiché visita ogni nodo esattamente una volta.
- **Ispezione del grado (`grado`):** Restituisce il numero di figli non nulli di un nodo (0, 1 o 2). Complessità: $O(1)$. **Questa complessità vale solo per l'albero binario**.
- **Accesso al padre (`padre`):** Sfrutta il puntatore esplicito `parent`. Complessità: $O(1)$.

3 Algoritmi di Visita di un Albero

Una **visita** consiste nell'esaminare tutti i nodi dell'albero esattamente una volta.

3.1 1. Visite in Profondità (DFS – Depth-First Search)

Possono essere implementate sia ricorsivamente (sfruttando implicitamente la pila di sistema dei record di attivazione) sia iterativamente (gestendo esplicitamente una `Pila / Stack`).

Si distinguono tre principali varianti basate sull'ordine in cui si visita la radice rispetto ai sottoalberi:

- **Preordine (Radice → Sinistra → Destra):** Utilizzata per duplicare o stampare la struttura gerarchica.
- **Simmetrica / In-order (Sinistra → Radice → Destra):** Fondamentale nei BST poiché visita i nodi in ordine crescente.
- **Postordine (Sinistra → Destra → Radice):** Essenziale per la deallocazione sicura della memoria (`free_tree`), poiché elimina prima i figli e poi il padre, evitando perdite di riferimento (*memory leaks*).

Complessità DFS: Temporale $O(n)$, Spaziale $O(h)$ (dove h è l'altezza dell'albero).

3.2 2. Visita in Ampiezza (BFS – Breadth-First Search)

Visita l'albero livello per livello (dalla radice verso il basso) sfruttando una struttura dati a **Coda (Queue)** secondo la politica FIFO.
Complessità BFS: Temporale $O(n)$, Spaziale $O(w)$ (dove w è la larghezza massima dell'albero).

4 Alberi Binari di Ricerca (BST – Binary Search Trees)

Un utilizzo fondamentale degli alberi binari è la realizzazione di dizionari efficienti ordinati per chiave.

4.1 Proprietà del BST

Per ogni nodo u dell'albero:

- Tutte le chiavi nel sottoalbero sinistro sono **minori** della chiave di u.
- Tutte le chiavi nel sottoalbero destro sono **maggiori** della chiave di u.

4.2 Operazioni sui BST

- **Ricerca (`lookup`):** Sfrutta il confronto delle chiavi per scendere selettivamente a sinistra o a destra. Complessità: $O(h)$ (da $O(\log n)$ nel caso bilanciato a $O(n)$ nel caso degenerato).
- **Minimo e Massimo (`min / max`):** Si ottengono seguendo sempre i puntatori rispettivamente a sinistra o a destra fino alle foglie. Complessità: $O(h)$.
- **Inserimento (`insert_bst`):** Inserisce un nuovo elemento preservando l'ordinamento delle chiavi e aggiornando i collegamenti bidirezionali. Complessità: $O(h)$.

5 Riepilogo delle Complessità

Operazione / Algoritmo	**Struttura Ausiliaria**	**Complessità Temporale**	**Complessità Spaziale**
Creazione nodo / Padre / Grado	Nessuna	$O(1)$	$O(1)$
Conteggio nodi (`num_nodi`)	Ricorsione (Call Stack)	$O(n)$	$O(h)$
Visite DFS (Pre/Sim/Post)	Pila / Stack	$O(n)$	$O(h)$
Visita in Ampiezza (BFS)	Coda / Queue	$O(n)$	$O(w)$
Ricerca / Min / Max / Inserimento BST	Nessuna (o ricorsione)	$O(h)$	$O(h)$ (ricorsione) / $O(1)$ (iterativo)

Nota: h rappresenta l'altezza dell'albero e w la larghezza massima.